CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati
saradhi@iitg.ac.in

Week #09: Exceptions

Exceptions - 01

- C++ exceptions are a feature of the language that allow to handle errors or unusual conditions in a program effectively.
- They help deal with runtime errors by enabling transfer control from one part of a program to another
- They facilitating error recovery without interrupting the flow of the program
- Errors are managed using return codes, flags, or error messages (populate one example for each)
- These methods are harder to maintain
- Exception handling provides a way to manage errors separately from normal code execution, making programs more robust and readable.

Exceptions - 02

- **Key elements of exceptions**
  - **Try block - The try block contains the code that may potentially throw an exception.**
  - **Throw statement - Signals an error or exceptional situation. It throws an exception object of any type.**
  - **Catch block - Code to handle the exception thrown in the try block. Multiple catch blocks can be used to handle different types of exceptions.**

Exceptions - 03

- **Syntax**

```
try
{

}
catch (input argument 1)
{

}
catch (input argument 2)
{

}
```

Exceptions - 04

- try Block: When a try block is executed, the program runs the code within it.
- Throwing an Exception: If an error or an unusual condition occurs, the throw statement is executed.
- catch Block: The catch block that matches the type of the thrown exception catches the exception.
- Control is transferred from the throw to the catch block, and the normal flow of the program is interrupted.
- Handling the Exception: The code in the catch block handles the exception. Once the catch block finishes executing, control is passed to the code after all catch blocks.

Exceptions - 05

```
#include <iostream>
using namespace std;
int main()
{
  try
  {
    int a = 10, b = 0;
    if ( b == 0 )
    {
      throw "Divided by zero";
    }
    cout << a/b << endl;
  }
  catch (const char *e)
  {
    cout << "Caught an exception " << e << endl;
  }
  return 0;
}
```

The throw statement throws char pointer, and it is caught by the catch block that handles const char * type exceptions.

Exceptions - 06

- The throw statement is used to signal that an exception has occurred.
- One can throw any data type (in-built data types, objects of standard classes, or user-defined classes).

```
throw "Const char pointer";

throw 404; // integert

throw 404.404 //float

throw runtime_error("Run time errror");
```

Exceptions - 07

- The catch block is used to catch various "types" of throws!

```
catch (const char* e)
{
  // Handle string exceptions
}
catch (int e)
{
  // Handle integer exceptions
}
catch (float e)
{
  // Handle integer exceptions
}
```

Exceptions - 08

- **Generic catch - It is helpful when you do not know the type of the exception or want to handle all exceptions in a single block.**

```
catch (...)
{
  // Handle any exception
}
```

Examples

Example - 01

```
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    try
    {
       throw 404;
    }
    catch (int e)
    {
      cout << "Caught an integer exception: " << e << endl;
    }
    return 0;
}
```

Example - 02

```cpp
#include <iostream>
using namespace std;

int main(int argc, char *argv[])
{
    try
    {
        throw 'A';
    }
    catch (int e)
    {
        cout << "Caught an integer exception: " << e << endl;
    }
    catch (char e)
    {
        cout << "Caught a character exception: " << e << endl;
    }
    return 0;
}
```

Example - 03

```cpp
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    try
    {
        throw 3.14;
    }
    catch (...)
    {
        cout << "Caught an unknown exception!" << endl;
    }
    return 0;
}
```

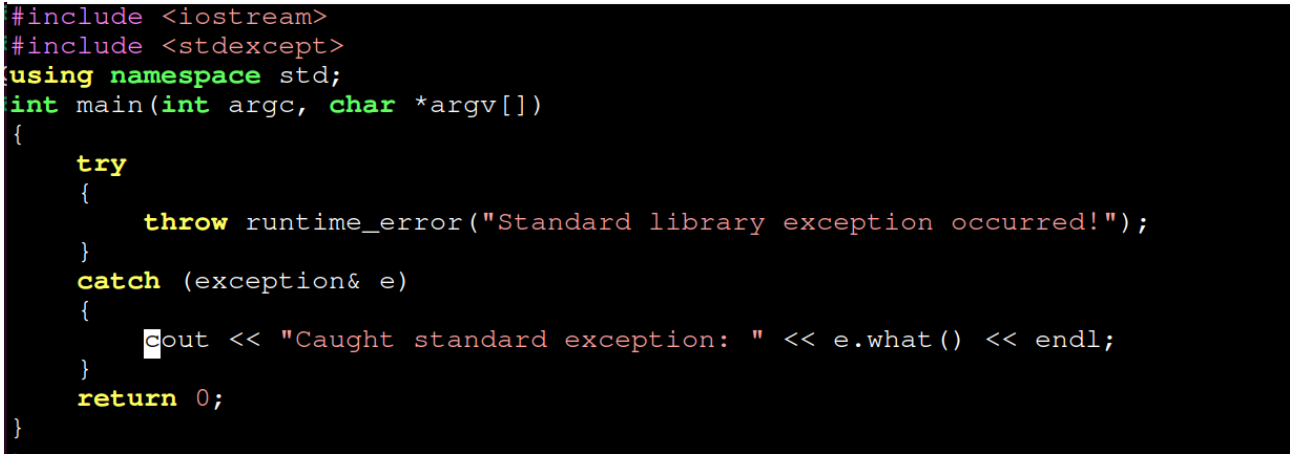
Example - 04

```
#include <iostream>
using namespace std;
class CustomException
{
public:
    CustomException()
    {
      cout << "CustomException Constructor";
    }
    const char* what() const
    {
        return "Custom Exception occurred!";
    }
};
int main(int argc, char *argv[])
{
    try
    {
        throw CustomException();
    }
    catch (CustomException& e)
    {
        cout << e.what() << endl;
    }
    return 0;
}
```

Example – 05 – re-throw

```cpp
#include <iostream>
using namespace std;
void func()
{
    try
    {
        throw "Error in func!";
    }
    catch (const char* e)
    {
        cout << "Caught inside func: " << e << endl;
        throw;
    }
}

int main(int argc, char *argv[])
{
    try
    {
        func();
    }
    catch (const char* e)
    {
        cout << "Caught in main: " << e << endl;
    }
    return 0;
}
```

Example – 06 – standard library exception



```cpp
#include <iostream>
#include <stdexcept>
using namespace std;
int main(int argc, char *argv[])
{
    try
    {
        throw runtime_error("Standard library exception occurred!");
    }
    catch (exception& e)
    {
        cout << "Caught standard exception: " << e.what() << endl;
    }
    return 0;
}
```

Example – 07 – Nested blocks

```cpp
#include <iostream>
using namespace std;
int main(int argc, char *argv[])
{
    try
    {
        try
        {
            throw "Nested error!";
        }
        catch (const char* e)
        {
            cout << "Caught inside nested try: " << e << endl;
            throw;
        }
    }
    catch (const char* e)
    {
        cout << "Caught in outer try: " << e << endl;
    }
    return 0;
}
```

Example – 08 – Strings

```cpp
#include <iostream>
#include <string>
using namespace std;
int main(int argc, char *argv[])
{
    try
    {
        throw string("String exception!");
    }
    catch (string& e)
    {
        cout << "Caught a string exception: " << e << endl;
    }
    return 0;
}
```

Example – 09 – conditional exception

```cpp
#include <iostream>
using namespace std;

void divide(int a, int b)
{
    if (b == 0) {
        throw invalid_argument("Division by zero!");
    }
    cout << "Result: " << a / b << endl;
}

int main(int argc, char *argv[])
{
    try
    {
        divide(10, 0);
    }
    catch (const invalid_argument& e)
    {
        cout << "Caught exception: " << e.what() << endl;
    }
    return 0;
}
```

The function divide() throws a invalid_argument exception if the second parameter is zero, preventing division by zero.

This exception is then caught in the main.

Exceptions - 09

- Standard exceptions are defined in <stdexcept> header file
- Important standard exceptions are
- **`exception`**: Base class for all standard exceptions.
- **`runtime_error`**: Errors that occur at runtime.
- **`logic_error`**: Errors in program logic, detectable before runtime.
- **`out_of_range`**: Accessing an element outside the valid range.

Run time errors

Example – 09 – Strings

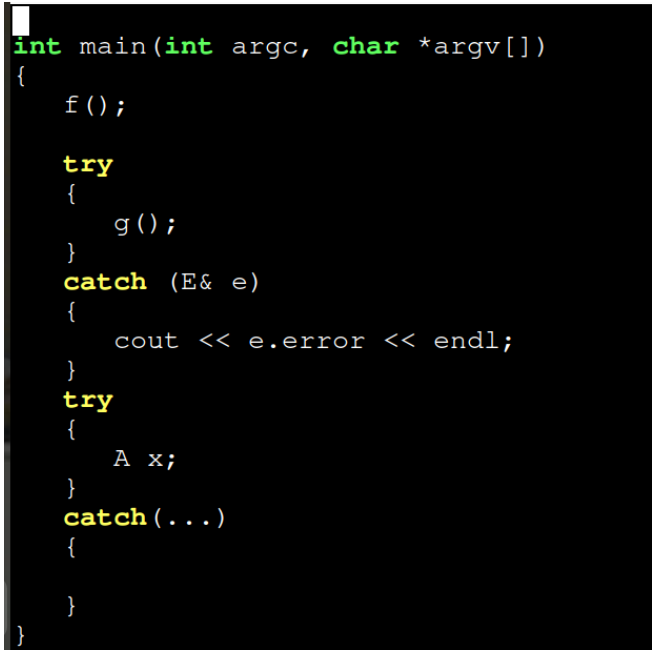
```cpp
#include <iostream>
#include <stdexcept>
using namespace std;
void allocateMemory()
{
    int *arr;
    try
    {
        arr = new int[100];
        throw runtime_error("Error during memory operation");
    }
    catch (...)
    {
        delete[] arr;
        throw;
    }
}
int main(int argc, char *argv[])
{
    try
    {
        allocateMemory();
    }
    catch (exception& e)
    {
        cout << "Caught exception: " << e.what() << endl;
    }
    return 0;
}
```

Example – 10

```
#include <iostream>
using namespace std;

class E
{
   public:
      const char* error;
      E(const char* arg) : error(arg)
      {

      }
};

class A
{
   public:
      int i;

      A() try : i(0)
      {
         throw E("Exception thrown in A()");
      }
      catch (E& e)
      {
         cout << e.error << endl;
      }
};
```

```
void f() try
{
   throw E("Exception thrown in f()");
}
catch (E& e)
{
   cout << e.error << endl;
}

void g()
{
   throw E("Exception thrown in g()");
}
```

Example – 10



```
int main(int argc, char *argv[])
{
   f();

   try
   {
      g();
   }
   catch (E& e)
   {
      cout << e.error << endl;
   }
   try
   {
      A x;
   }
   catch(...)
   {

   }
}
```

Logic Errors

Example – 11

```
void push(const T& value)
{
  if (topIndex == capacity - 1)
  {
    throw logic_error("Stack overflow: Cannot push!");
  }
  data[++topIndex] = value;
}
```

```
T pop()
{
  if (topIndex == -1)
  {
    throw logic_error("Stack underflow: Cannot pop");
  }
  topIndex--;
}
```

Example – 11

```
try
{
  Stack<int> stack(3);
  stack.push(10); stack.push(20); stack.push(30);
  stack.push(40);
}
catch (const logic_error& e)
{
  cout << "Logic error caught: " << e.what() << endl;
}
```

Example – 11

```
try
{
  Stack<int> stack(3);
  stack.pop();
}
catch (const logic_error& e)
{
  cout << "Logic error caught: " << e.what() << endl;
}
```

out_of_range Exception

Example – 12

```
try
{
  string s1 = "Hello";
  cout << "Character at index 10: " << s1.at(10) << endl;
}
catch (const out_of_range& e)
{
  cout << "Out of Range Exception: " << e.what() << endl;
}
```

Exceptions

Example – 13

```
class Base: public exception
{
  public:
    virtual const char* what() const noexcept override
    {
      return "Base class Exception occurred";
    }
};
```

```
class Derived1 : public Base
{
  public:
    const char* what() const noexcept override
    {
      return "Derived1 exception occurred";
    }
};
```

Example – 13

```
class Derived2 : public Base
{
  public:
    const char* what() const noexcept override
    {
      return "Derived2 exception occurred";
    }
};
```

```
try
{
  throw Derived1();
}
catch (Base& e)
{
  cout << "Caught an exception: " << e.what() << std::endl;
}
```

Example – 13

```
try
{
  throw Derived2();
}
catch (Base& e)
{
  cout << "Caught an exception: " << e.what() << std::endl;
}
```

Base class is derived from exception class

Base class overrides the what() method to provide a description of the error.

`Derived1` and `Derived2` are derived from `Base class`

Each derived class overrides the `what()` method to provide a specific error message.

Both exceptions are caught using a `catch` block for `Base`

Example – 13

```
try
{
  throw Derived1();
}
Catch (Derived1& d)
{
 cout << "Caught an exception: " << d.what() << endl;
}
catch (Base& e)
{
  cout << "Caught an exception: " << e.what() << endl;
}
```

This catches derived1 class exception

Stack Unwinding

Stack Unwinding - 01

- When an exception is thrown and control passes from a try block to a handler, the C++ run time calls destructors for all automatic objects constructed since the beginning of the try block.
- This process is called stack unwinding.
- The automatic objects are destroyed in reverse order of their construction.
- Automatic objects are local objects that have been declared auto or register, or not declared static or extern.
- An automatic object x is deleted whenever the program exits the block in which x is declared
- If an exception is thrown during construction of an object consisting of subobjects or array elements, destructors are only called for those subobjects or array elements successfully constructed before the exception was thrown.
- A destructor for a local static object will only be called if the object was successfully constructed.
- If during stack unwinding a destructor throws an exception and that exception is not handled, the terminate() function is called.

Example – 14

```
class E
{
  public:
    const char* message;
    E(const char* arg) : message(arg) { }
};
```

```
class A
{
  public:
    A() { cout << "In constructor of A" << endl; }
    ~A()
      {
      cout << "In destructor of A" << endl;
      throw E("Exception thrown in ~A()");
    }
};
```

```
class B
{
    public:
    B() { cout << "In constructor of B" << endl; }
    ~B() { cout << "In destructor of B" << endl; }
};

int main(int argc, char *argv[])
{
  try
    {
    cout << "In try block" << endl;
    A a;
    B b;
    throw("Exception thrown in main()");
  }
  catch (const char* e)
    {
    cout << "Exception: " << e << endl;
  }
  catch (...)
    {
    cout << "exception caught in main()" << endl;
  }
  cout << "Resume execution of main()" << endl;
}
```

Stack Unwinding - 02

- Two automatic objects are created: a and b.
- The try block throws an exception of type const char*.
- The handler catch (const char* e) catches this exception.
- The C++ run time unwinds the stack, calling the destructors for a and b in reverse order of their construction.
- The destructor for a throws an exception.
- Since there is no handler in the program that can handle this exception, the C++ run time calls terminate().

Stack Unwinding - 03

```
class D
{
  public:
    D()
    {
      cout << "D().\n";
    }
    D(char) try: d_char(55)
    {
      cout << "D(char). Throws.\n";
      throw 0;
    }
    catch(...)
    {
      cout << "D(char).Catch block.\n";
    }
    D(int i):d_int(i)
    {
      cout << "D(int).\n";
    }
    ~D()
    {
      cout << "~D().\n";
    }
};
```

```
int main(int argc, char *argv[])
{
  D d;
};
```

Exceptions

Exceptions - 02

- In a handler, you specify the types of exceptions that it may process.
- The C++ run time, together with the generated code, will pass control to the first appropriate handler that is able to process the exception thrown.
- When this happens, an exception is caught.
- A handler may rethrow an exception so it can be caught by another handler.
- The exception handling mechanism is made up of the following elements:
    - Try block
    - Catch block
    - Throw expressions
    - Exception Specifications

Try Catch Block - 01

```
#include <iostream>
using namespace std;
int main()
{
  try
  {
    // Throwing an integer
    throw 404;
  }
  catch (int e)
  {
    // Catching the integer
    cout << "Caught an integer exception:" << e << endl;
  }
  return 0;
}
```

The throw statement throws an integer, and it is caught by the catch block that handles int type exceptions.

Try Catch Block - 02

```
#include <iostream>
using namespace std;
int main()
{
  try
  {
    throw 'A'; // Throwing a character
  }
  catch (int e)
  { // Will not catch the character
    std::cout << "Caught an integer: " << e << std::endl;
  } catch (char e)
  { // Catching the character
    cout << "Caught a character exception: " << e << endl;
  }
```

The throw statement throws an integer, and it is caught by the catch block that handles int type exceptions.

Exceptions As Classes And Objects

Exceptions as classes & objects

```
// add with overflow checking

struct Int_overflow {    // a class for integer overflow exception handling
    const char* op;
    int operand1, operand2;
    Int_overflow(const char* p, int q, int r):
        op(p), operand1(q), operand2(r) { }
};

int add(int x, int y)
{
    if (x > 0 && y > 0 && x > MAXINT - y
     || x < 0 && y < 0 && x < MININT + y)
        throw Int_overflow("+", x, y);

    // If we get here, either overflow has
    // been checked and will not occur, or
    // overflow is impossible because
    // x and y have opposite sign

    return x + y;
}
```

```
main()
{
    int a, b, c;

    cin >> a >> b;
    try {
        c = add(a,b);
    }
    catch (Int_overflow e) {
        cout << "overflow " << e.op << '('
             << e.operand1 << ',' << e.operand2
             << ")\n";
    }

    // and so on
}
```

Grouping of Exceptions

Grouping Exceptions

- Exceptions often fall naturally into families.
- One could imagine a Matherr exception that includes Overflow, Underflow, and other possible exceptions.
- One way of doing this might be simply to define Matherr as a class whose possible values include
- Overflow and the others:

```
enum Matherr { Overflow, underflow, Zerodivide, /* ... */ };
```

Grouping Exceptions

```
try {
    f();
}
catch (Matherr m) {
    switch (m) {
    case Overflow:
        // ...
    case Underflow:
        // ...
    // ...
    }
    // ...
}
```

In other contexts, C++ uses inheritance and virtual functions to avoid this kind of switch on a type field.

It is possible to use inheritance similarly to describe collections of exceptions too.

Grouping Exceptions

```
class Matherr { };
class Overflow: public Matherr { };
class Underflow: public Matherr { };
class Zerodivide: public Matherr { };
// ...
```

```
try {
    f();
}
catch (Overflow) {
    // handle Overflow or anything derived from Overflow
}
catch (Matherr) {
    // handle any Matherr that is not Overflow
}
```

Here an Overflow is handled specifically

All other Matherr exceptions will be handled by the general case.

There are cases where an exception can reasonably be considered to belong to two or more groups.

```
class network_file_err : public network_err , public file_system_err { };
```

Naming Current Exception

Naming Exceptions

- We name the current exception analogously to naming the argument of a function

```
try { /* ... */ } catch (zaq e) { /* ... */ }
```

- This is a declaration of e with its scope identical to the body of the exception handler.
- The idea is that e is created using the copy constructor of class zaq before entering the handler
- Where there is no need to identify the particular exception, there is no need to name the exception object

```
try { /* ... */ } catch (zaq) { /* ... */ }
```

Order of Matching Exceptions

- **Given the following series of catch blocks, how to determine the order of matching?**

```
try {
    // ...
}
catch (ibuf) {
    // handle input buffer overflow
}
catch (io) {
    // handle any io error
}
catch (stdlib) {
    // handle any library exception
}
catch (...) {
    // handle any other exception
}
```

The type in a catch clause matches if either

it directly refers to the exception thrown

is of a base class of that exception

if the exception thrown is a pointer and the exception caught is a pointer to a base class of that exception.

Naming Exceptions

- **Such a re-throw operation is valid only within a handler or within a function called directly or indirectly from a handler.**

```
try { /* ... */ } catch (zaq e) { /* ... */ }
```

- This is a declaration of e with its scope identical to the body of the exception handler.
- The idea is that e is created using the copy constructor of class zaq before entering the handler
- Where there is no need to identify the particular exception, there is no need to name the exception object

```
try { /* ... */ } catch (zaq) { /* ... */ }
```

Questions?

Thank You